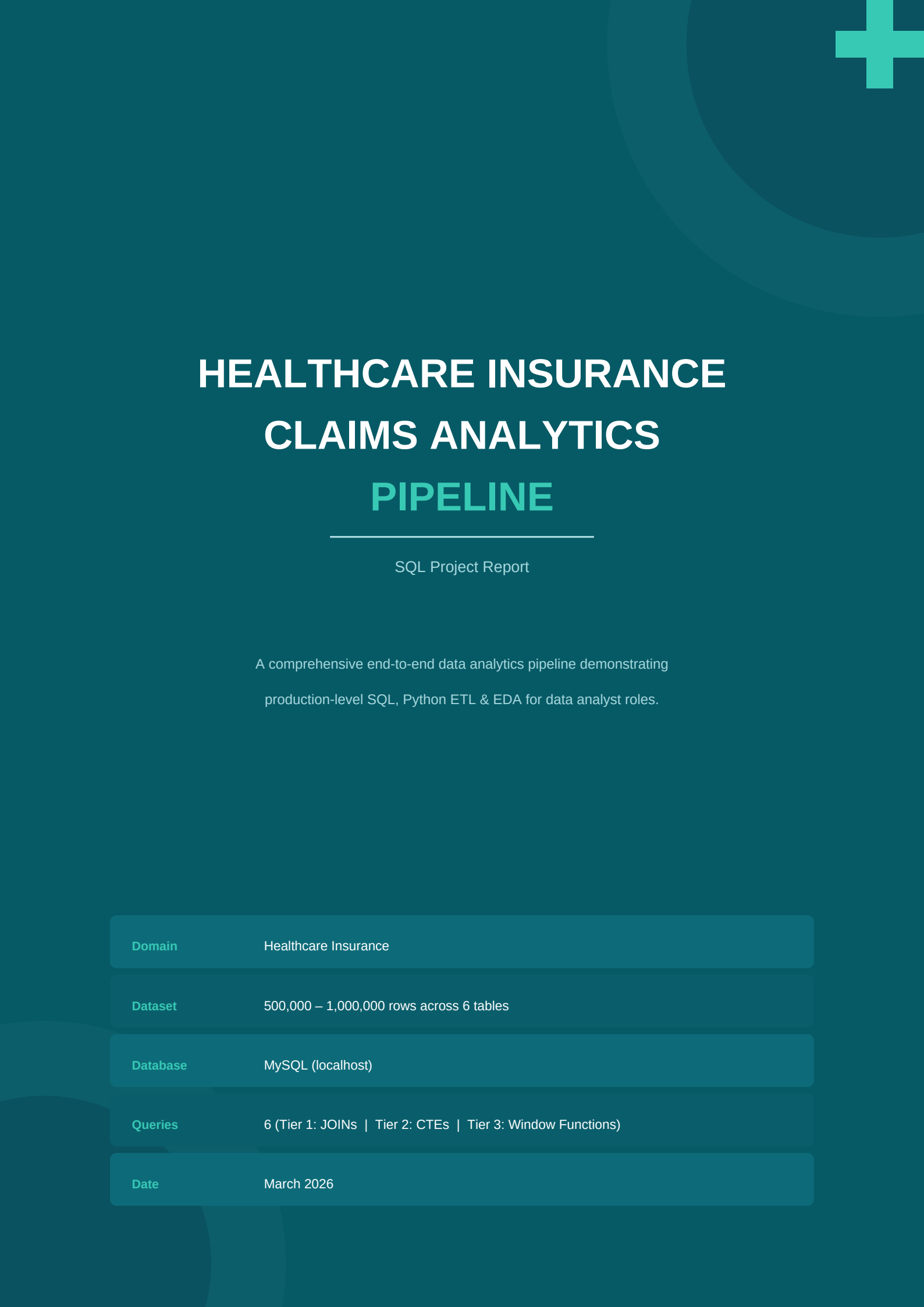




HEALTHCARE INSURANCE CLAIMS ANALYTICS PIPELINE

SQL Project Report

A comprehensive end-to-end data analytics pipeline demonstrating
production-level SQL, Python ETL & EDA for data analyst roles.

Domain	Healthcare Insurance
Dataset	500,000 – 1,000,000 rows across 6 tables
Database	MySQL (localhost)
Queries	6 (Tier 1: JOINS Tier 2: CTEs Tier 3: Window Functions)
Date	March 2026

1. Project Overview

This project is an end-to-end data analytics pipeline built on a synthetic Healthcare Insurance Claims dataset. It demonstrates the full lifecycle of a data analyst's work — from database schema design and ETL pipeline development to SQL analysis and Python-based exploratory data analysis (EDA).

The project was built as a centrepiece portfolio piece to demonstrate competency across SQL, Python, data engineering, and analytical thinking — skills directly relevant to data analyst roles.

Attribute	Detail
Domain	Healthcare Insurance
Dataset Size	500,000 – 1,000,000 rows across 6 tables
Database	MySQL (localhost)
Pipeline Type	ETL (Extract, Transform, Load)
Intermediate Format	Apache Parquet
Languages	Python, SQL
Python Libraries	Pandas, NumPy, Faker, SQLAlchemy, PyMySQL, Matplotlib, Seaborn
EDA Environment	Google Colab
Version Control	GitHub (PrithuVerma)

2. Objectives

- Design a normalised, production-realistic relational schema for insurance claims data
- Build a structured ETL pipeline in Python that generates, validates, and loads data automatically
- Use Parquet as an intermediate format for efficient columnar storage
- Write advanced SQL queries (CTEs, window functions, complex joins) to answer business questions
- Perform exploratory data analysis in Python to surface insights across patients, claims, and hospitals
- Package the project professionally for GitHub with a clean folder structure

3. Database Schema Design

The schema consists of 6 normalised tables designed to model the end-to-end lifecycle of an insurance claim — from patient registration through policy issuance, claim submission, procedure recording, and final adjudication.

Table	Rows (approx.)	Description
Patients	750,000	Patient demographics — DOB, gender, contact details
Hospitals	130	Hospital registry — name, city, state, type (Public/Private/Clinic)
Policies	300,000	Insurance policies — type, coverage amount, premium, validity dates
Claims	300,000	Claim transactions — amounts, dates, status (Pending/Approved/Denied)
Procedures	700,000+	Medical procedures per claim — category, name, cost, date

Table	Rows (approx.)	Description
Claim Reviews	50,000	Adjudication records — decision, reviewer, rejection reason

Key Design Decisions

- **patient_id removed from Claims** — derivable via Policy → Patient join, avoiding data integrity risk
- **age removed from Patients** — calculated from dob at query time to avoid stale data
- **phone_num stored as VARCHAR** — BIGINT drops leading zeros; phone numbers are never used arithmetically
- **city and state as separate columns in Hospitals** — enables clean regional SQL analysis
- **rejection_reason in Claim Reviews** — critical for denial pattern analysis
- **procedure_category as ENUM** — enables aggregation by type (Surgery, Diagnostic, Therapy, Consultation)

Table Relationships

- Patients → Policies (1-to-many): one patient can hold multiple policies
- Policies → Claims (1-to-many): one policy can have multiple claims filed against it
- Hospitals → Claims (1-to-many): one hospital can submit multiple claims
- Claims → Procedures (1-to-many): one claim can contain multiple procedures
- Claims → Claim Reviews (1-to-many): one claim can go through multiple review cycles

4. ETL Pipeline

The ETL pipeline is a structured Python project consisting of 5 files. Running `pipeline.py` executes the full Extract → Transform → Load sequence automatically with no manual intervention.

File	Role	Key Responsibility
<code>config.py</code>	Configuration	DB credentials, file paths, row counts, ENUM value lists
<code>generate_data.py</code>	Extract	Generates synthetic data using Faker (<code>en_IN</code>) and NumPy distributions, saves to <code>data/raw/</code> as Parquet
<code>transform.py</code>	Transform	Validates and fixes data quality issues, saves to <code>data/processed/</code> as Parquet
<code>load.py</code>	Load	Reads processed Parquet files, pushes to MySQL via SQLAlchemy (<code>if_exists='replace'</code>)
<code>pipeline.py</code>	Orchestrator	Chains all three stages in sequence — single entry point for the entire pipeline

Extract — Synthetic Data Generation

Data was generated entirely synthetically using Faker with the `en_IN` (India) locale for realistic Indian names, addresses, and geography. NumPy statistical distributions were applied to ensure the data behaves realistically rather than uniformly randomly.

- **Claim amounts:** exponential distribution (`scale=50,000`) — right-skewed, most claims small, few very large
- **Procedure counts per claim:** weighted random `[1,2,3,4]` with weights `[40,35,20,5]%`
- **Coverage amounts:** range-based by policy type (e.g., Critical Illness: Rs. 5,00,000–Rs. 50,00,000)
- **Phone numbers:** custom generated — 10 digits, first digit always 6/7/8/9 (Indian mobile standard)
- **IDs:** zero-padded prefixed format (`PAT0000001`, `CLM0000001`) for readability and consistent length
- **Chained ID passing:** each generator receives parent table IDs as input, guaranteeing FK integrity

Transform — Validation Strategy

Transforms follow a **fix-vs-drop** strategy: rows with correctable issues are fixed using existing data; rows with fundamental contradictions are dropped.

Validation Rule	Strategy	Rationale
<code>approved_amount > coverage_amount</code>	Fix — cap at <code>coverage_amount</code>	Can be corrected using existing column
<code>procedure_date < claim_date</code>	Fix — set <code>procedure_date = claim_date</code>	Procedure must occur on or after claim date
<code>review_date < claim_date</code>	Fix — <code>review_date = claim_date + 1 day</code>	Review cannot precede the claim
Expired policy with future <code>end_date</code>	Fix — set <code>end_date = today</code>	Status and date must be consistent
<code>claim_date</code> outside policy validity	Drop	Cannot determine which date is incorrect — row is contradictory
Invalid ENUM values	Drop	Values outside defined categories are unrecoverable
Orphaned foreign keys	Drop	Referential integrity cannot be restored without inventing data

5. SQL Analysis

Six analytical queries were written against the loaded MySQL database, progressively increasing in complexity from simple aggregations to multi-CTE window function queries.

Query	Business Question	SQL Concepts Used
Q1	Approved claim amounts vs total policy coverage	JOIN, SUM, MAX, GROUP BY, WHERE
Q2	Distribution of claim statuses (Pending/Approved/Denied)	GROUP BY, COUNT, CASE WHEN
Q3	Procedure categories by claim volume and approval rate	JOIN, GROUP BY, COUNT, CASE WHEN, calculated rate
Q4	Hospital ranking by claim volume within each hospital type	CTE, JOIN, COUNT, RANK() OVER (PARTITION BY)
Q5	Monthly and quarterly P&L — premium collected vs aid paid	CTE chaining, YEAR(), QUARTER(), MONTH(), NULLIF(), CASE WHEN
Q6	Most common claim rejection reasons	GROUP BY, COUNT, ORDER BY, WHERE IS NOT NULL

Notable Query — Q4: Hospital Rankings with Window Functions

This query demonstrates CTE composition combined with **RANK()** window function partitioned by hospital type — a common pattern in real analytics work.

```
WITH Hospital_Claims AS (  
  SELECT h.name, h.type, COUNT(c.claim_id) AS claims_provided  
  FROM claims c LEFT JOIN hospitals h ON h.hospital_id = c.hospital_id  
  GROUP BY h.name, h.type  
)  
SELECT name, type, claims_provided,  
  RANK() OVER (PARTITION BY type ORDER BY claims_provided DESC) AS ranking  
FROM Hospital_Claims;
```

Notable Query — Q5: P&L Analysis with Chained CTEs

The P&L query uses two chained CTEs — the first aggregates premium and aid by time period and policy type, the second applies business logic (Profit/Loss classification and ratio calculation) on top of the first. **NULLIF()** prevents division-by-zero errors.

6. Python EDA

Exploratory data analysis was performed in Google Colab reading directly from the processed Parquet files stored in Google Drive. The EDA answers the same business questions as the SQL analysis but from a Python perspective, demonstrating versatility across tools.

Analysis	Method	Key Finding
Age distribution of patients	pd.cut() bins, bar chart	Distribution across Young Adult, Adult, Middle-aged, Senior segments
Claim amount distribution	sns.histplot(kde=True), describe()	Right-skewed as designed — confirms exponential distribution
Avg claim by procedure category	merge() + groupby() + mean()	Surgery highest avg cost; Consultation lowest

Analysis	Method	Key Finding
Approval rate by gender	groupby() + value_counts(normalize=True)	Minimal disparity across gender groups
Top states by claims filed	groupby() + sum() + sort_values()	Regional concentration visible across Indian states
Monthly claim trend over years	dt accessor + groupby() + line chart	Claim volume trend over 2018–2024 period
Avg processing time by decision	timedelta calculation + groupby()	Denied claims reviewed faster than approved ones
High-risk policy flagging	Boolean mask on approved/coverage ratio	Flags policies where approved > 90% of coverage
Hospitals with high denial rates	groupby() + comparison to mean	Outlier hospitals identified for potential audit
Repeat claimant cohort analysis	groupby() + year extraction + merge()	Percentage of Year 1 claimants who returned in Year 2

7. Project Structure

The project follows a clean, modular folder layout separating concerns across ETL, data storage, SQL, and analysis layers.

Medical_Insurance/	
-- ETL/	
-- config.py	# DB credentials, constants, ENUM values
-- generate_data.py	# Extract -- synthetic data generation
-- transform.py	# Transform -- validation and cleaning
-- load.py	# Load -- push to MySQL via SQLAlchemy
`-- pipeline.py	# Orchestrator -- runs full ETL sequence
-- data/	
-- raw/	# Parquet files post-extraction
`-- processed/	# Parquet files post-transformation
-- sql/	
-- schema.sql	# CREATE TABLE statements
`-- analysis.sql	# All 6 analytical queries
-- Health_Insurance_Analysis.ipynb	
# Python EDA notebook (Google Colab)	
-- schema.png	
# ER diagram	
`-- README.md	
# Setup instructions and project documentation	

8. Key Learnings

• ETL Pipeline Design — Build for Idempotency from Day One

Designing the pipeline with if_exists='replace' and parent-to-child load ordering ensured the entire pipeline could be re-run safely at any time without producing duplicate or orphaned records. Retrofitting idempotency into an existing pipeline is far more costly than baking it in from the start. FK integrity constraints must also be planned at schema design time — not patched in later.

• Parquet vs CSV — Columnar Storage Wins at Scale

At 700,000+ rows, switching from CSV to Apache Parquet yielded significant performance gains. Parquet's columnar format means queries that touch only a few columns skip entire column chunks on disk. For analytical workloads where most queries are column-selective rather than row-selective, this translates to faster reads, lower memory usage, and smaller file sizes due to built-in compression.

• Fix vs Drop — A Principled Data Quality Strategy

Not all data quality issues are equal. Rows with correctable inconsistencies (e.g., approved amount slightly exceeding coverage) can be fixed in-place using existing data. But rows with fundamental contradictions (e.g., claim_date outside policy validity range) cannot be corrected without inventing data — and must be dropped. Establishing this distinction upfront prevents silent corruption of downstream analysis.

• Statistical Realism — Distributions Matter in Synthetic Data

Uniformly random synthetic data produces unrealistic flat distributions that don't stress-test analytics logic properly. Applying an exponential distribution for claim amounts (right-skewed: most claims small, few very large) and weighted random choices for procedure counts per claim produced data that behaves like real insurance claims — validating that the EDA findings and SQL aggregations would generalise to production data.

• SQL Execution Order — The Mental Model that Unlocks Query Debugging

Understanding that SQL processes clauses in the order FROM -> JOIN -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY is the single most important debugging insight for complex queries. It explains why column aliases defined in SELECT cannot be referenced in WHERE, why aggregate functions cannot appear in WHERE (use HAVING instead), and why JOIN conditions must reference column names rather than aliases.

• Window Functions vs GROUP BY — Two Different Aggregation Paradigms

GROUP BY collapses multiple rows into a single summary row, discarding row-level detail. Window functions (RANK, ROW_NUMBER, LAG, SUM OVER) compute aggregations across a partition of rows while preserving every individual row in the result set. This distinction is critical for ranking queries: GROUP BY cannot rank hospitals within type while retaining each hospital's name and claim count — PARTITION BY can.

• CTE Chaining — Clean Multi-Step Logic Without Nested Subqueries

Chained CTEs allow the second CTE to reference the first as if it were a base table, enabling complex multi-step transformations to be written as a readable sequence of named steps rather than deeply nested subqueries. In the P&L query, the first CTE aggregated premium and aid by time period; the second CTE applied Profit/Loss classification on top — keeping each logical step isolated and testable.

• NULLIF() for Safe Division — Always Protect Your Denominators

Division-by-zero is a silent runtime error in SQL that can crash entire reports. Wrapping denominators in NULLIF(column, 0) returns NULL instead of an error when the denominator is zero, which then propagates gracefully through ratio calculations. This pattern is especially important in P&L and approval rate queries where some policy types or time periods may have zero premium or zero claims in a given month.

• Vectorised Operations in Pandas — Never Loop Row by Row

Applying pd.cut() to bin 750,000 patient ages into age groups takes milliseconds as a vectorised operation. The equivalent Python for loop iterating row by row would take minutes. This principle — use Pandas built-in vectorised methods (apply sparingly, loops never) — is the single biggest performance lever in Python-based EDA at scale. The same applies to merge(), groupby(), and boolean masking for outlier flagging.

9. Skills Demonstrated

Database Design

- Designed a 6-table normalised relational schema from scratch, modelling the full insurance claim lifecycle
- Made deliberate data type decisions — VARCHAR for phone numbers, calculated age from DOB, ENUM for procedure categories
- Enforced referential integrity through FK relationships with parent-to-child dependency ordering
- Removed redundant columns (patient_id from Claims) to eliminate data integrity risks at the schema level

ETL Pipeline Development

- Built a fully automated, idempotent Extract → Transform → Load pipeline in Python with a single entry point
- Separated concerns cleanly across config, generation, validation, loading, and orchestration files
- Applied a principled fix-vs-drop validation strategy to handle 7 distinct data quality issue categories
- Used Parquet as the intermediate format for efficient columnar storage between pipeline stages

Python — Data Engineering

- Generated 1M+ rows of statistically realistic synthetic data using Faker (en_IN locale) and NumPy distributions
- Managed DataFrame I/O with Pandas — Parquet read/write, type casting, and memory-efficient operations
- Connected to MySQL via SQLAlchemy engine with idempotent table replacement and FK-respecting load order

- Engineered chained ID passing between generators to guarantee foreign key integrity across all 6 tables

Python — Exploratory Data Analysis

- Performed 10 distinct analyses covering demographics, financials, regional trends, and cohort behaviour
- Applied vectorised Pandas operations (pd.cut, groupby, merge, boolean masks) across 750K+ row DataFrames
- Identified outlier hospitals with above-average denial rates using statistical comparison to group mean
- Built a repeat claimant cohort analysis tracking Year 1 patients who returned in Year 2
- Visualised distributions, trends, and comparisons using Matplotlib and Seaborn (histplot, barplot, lineplot)

SQL — Intermediate

- Wrote multi-table JOIN queries navigating a 3-hop path (Claims → Policies → Patients) without direct FK
- Used CASE WHEN to calculate approval rates and classify P&L; outcomes inline within SELECT
- Applied date functions (YEAR, QUARTER, MONTH) for time-series aggregation across a 6-year claims history
- Filtered and aggregated denial reasons using WHERE IS NOT NULL with GROUP BY and ORDER BY

SQL — Advanced

- Wrote CTEs to modularise complex multi-step queries into readable, independently testable named blocks
- Chained two CTEs where the second references the first — enabling clean layered business logic application
- Applied RANK() window function with PARTITION BY hospital type to rank hospitals within each category
- Used NULLIF() to prevent division-by-zero errors in ratio and margin calculations across all financial queries

Analytical Thinking

- Translated 6 real business questions into SQL queries spanning aggregation, ranking, and profitability analysis
- Surfaced fraud signals by flagging policies where approved amount exceeded 90% of coverage
- Identified operational inefficiencies — denied claims reviewed significantly faster than approved ones
- Built a P&L framework comparing premium income against claim payouts by policy type and time period

Software Engineering

- Applied modular file structure with single-responsibility files — no monolithic scripts
- Centralised all constants (row counts, ENUM values, DB credentials) in config.py for easy configuration
- Protected all module entry points with if __name__ == '__main__' guards to support both import and direct execution
- Used print logging throughout the pipeline for transparent progress tracking without external dependencies

10. Future Scope

- **PowerBI dashboard** covering regional claim distribution, policy type breakdown, hospital performance, and 5-year P&L trends
- **Fraud detection model** using statistical outlier flagging on claim amounts and denial rates
- **Pipeline scheduling** using Apache Airflow for automated daily data refresh
- **Cloud migration** — AWS S3 for Parquet files and Amazon RDS for MySQL